Especificación de Requerimientos de Software (SRS): Detección de diferencias en escena usando estimación de fondo monocular

Marco Alonso Rodríguez Villegas
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
markorovi24@estudiantec.cr

I. Introducción

I-A. Propósito

El propósito de este documento es especificar, de manera completa, verificable y no ambigua, los requerimientos del software encargado de detectar diferencias entre dos escenas de un mismo espacio interior, capturadas con una cámara monocular en momentos distintos ("antes" y "después"). El documento sigue la estructura de la sección 9.6 del estándar ISO/IEC/IEEE 29148:2018 [1] y está dirigido al equipo de desarrollo, a la persona supervisora del proyecto y a cualquier evaluador técnico que deba validar el alcance funcional del sistema. Este documento describe exclusivamente las características, funciones y restricciones del producto de software a construir.

I-B. Alcance

El producto de software, denominado en adelante *Sistema de Detección de Diferencias por Estimación de Fondo Monocular* (en lo sucesivo, "el sistema"), recibirá como entrada dos imágenes RGB monoculares de un mismo espacio interior, tomadas en dos instantes distintos. A partir de dichas imágenes, el sistema deberá:

- Estimar la profundidad monocular de cada escena mediante un modelo de aprendizaje profundo preentrenado (p. ej. Depth Anything [3] o MiDaS [4]).
- Comparar los mapas de profundidad resultantes para determinar si existe una diferencia significativa entre ambas capturas.
- En caso de detectarse una diferencia, clasificarla como *desaparición* o *desplazamiento* de un objeto, calculando en este último caso la distancia de desplazamiento en centímetros.
- Identificar, mediante un Vision Transformer de segmentación semántica preentrenado (p. ej. SegFormer [5] o Mask2Former [6]), a qué categoría de objeto corresponde el cambio detectado.
- Presentar los resultados del análisis a la persona usuaria.

El alcance del sistema se limita a objetos ubicados en ambientes interiores, capturados con una cámara RGB monocular convencional (webcam o cámara de teléfono celular). El sistema no contempla el uso de sensores de profundidad dedicados (LiDAR, cámaras RGB-D), no realiza seguimiento de video en tiempo real (*tracking*) de múltiples cuadros, ni gestiona escenas exteriores. El sistema constituye un producto autocontenido; no forma parte de un sistema de mayor nivel ya existente, por lo que no se documentan interfaces con sistemas externos adicionales a las descritas en la sección III-C.

I-C. Resumen del producto

I-C1. Perspectiva del producto: El sistema es un producto de software independiente (*standalone*) que no reemplaza ni se integra con un sistema de vigilancia o de control de inventario preexistente; por el contrario, constituye una alternativa de bajo costo frente a los sensores de profundidad dedicados que actualmente se emplean para la verificación de cambios en espacios interiores. El sistema recibe únicamente imágenes capturadas por una cámara RGB monocular estándar y depende internamente de dos modelos de aprendizaje profundo preentrenados (uno de estimación de profundidad monocular y otro de segmentación semántica) ejecutados sobre el *framework* PyTorch [7]. La Figura 1 ilustra el contexto general de operación del sistema.

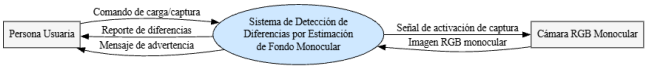


Figura 1. Diagrama de contexto del sistema de detección de diferencias, detallando los flujos de información con la Persona Usuaria y la Cámara RGB Monocular.

I-C2. Funciones del producto: De manera general, el sistema deberá ser capaz de:

1. Capturar o recibir dos imágenes monoculares RGB de una misma escena interior ("antes" y "después").
2. Ejecutar un modelo de estimación de profundidad monocular sobre ambas imágenes para reconstruir la distancia relativa al fondo de la escena.
3. Comparar los mapas de profundidad para determinar la existencia de diferencias entre las dos capturas.

4. Clasificar cada diferencia detectada como desaparición o desplazamiento de un objeto.
5. Calcular la distancia de desplazamiento, en centímetros, cuando corresponda.
6. Ejecutar un modelo de segmentación semántica para identificar la categoría del objeto asociado al cambio.
7. Reportar los resultados obtenidos a la persona usuaria de forma clara y comprensible.

I-C3. Características del usuario: El sistema está dirigido a personas usuarias con conocimientos técnicos básicos en el manejo de aplicaciones de escritorio o de línea de comandos, tales como personal de supervisión de espacios de trabajo, encargados de control de inventario o personal de mantenimiento de instalaciones. No se asume que la persona usuaria posea conocimientos de visión por computadora, aprendizaje profundo o procesamiento digital de imágenes; por lo tanto, la interfaz y los mensajes del sistema deben emplear lenguaje no técnico siempre que sea posible.

I-C4. Limitaciones: Las siguientes limitaciones condicionan las decisiones de diseño del desarrollador del producto:

- El sistema debe ejecutarse sobre hardware que cuente con una unidad de procesamiento gráfico (GPU) para que la inferencia de los modelos de aprendizaje profundo se realice en un tiempo razonable.
- El sistema debe desarrollarse en lenguaje Python, empleando el *framework* PyTorch [7], por ser el ecosistema donde se distribuyen los modelos preentrenados de estimación de profundidad monocular y de segmentación semántica considerados.
- El sistema únicamente puede emplear modelos preentrenados de estimación de profundidad monocular y de segmentación semántica disponibles públicamente (p. ej. Depth Anything, MiDaS, SegFormer, Mask2Former); no se contempla el entrenamiento de modelos desde cero.
- El desarrollo debe realizarse con una única cámara RGB monocular, sin sensores de profundidad dedicados.

I-D. Definiciones

- **Imagen monocular**: Imagen digital capturada por una única cámara, sin información estereoscópica ni de profundidad directa.
- **Estimación de profundidad monocular**: Técnica de visión por computadora que infiere, a partir de una única imagen RGB, un mapa de distancias relativas entre la cámara y los elementos de la escena.
- **Vision Transformer (ViT)**: Arquitectura de red neuronal basada en el mecanismo de atención (*transformer*), adaptada para el procesamiento de imágenes en tareas como clasificación o segmentación semántica.
- **Segmentación semántica**: Tarea de visión por computadora que asigna una etiqueta de categoría a cada píxel de una imagen, permitiendo identificar a qué objeto o superficie pertenece.
- **Inferencia**: Proceso de ejecutar un modelo de aprendizaje profundo ya entrenado sobre nuevos datos de entrada para obtener una predicción, sin modificar los parámetros del modelo.
- **GPU (Unidad de Procesamiento Gráfico)**: Hardware especializado en la ejecución paralela de operaciones matriciales, utilizado para acelerar el entrenamiento y la inferencia de modelos de aprendizaje profundo.
- **PyTorch**: *Framework* de código abierto para el desarrollo y la ejecución de modelos de aprendizaje profundo [7].
- **Mapa de profundidad**: Representación en la que cada píxel almacena un valor proporcional a la distancia estimada entre la cámara y el punto correspondiente de la escena.
- **Pipeline**: Secuencia ordenada de etapas de procesamiento (captura, inferencia, comparación, reporte) que transforma las imágenes de entrada en un resultado final.

I-E. Siglas y abreviaturas

- **SRS**: *Software Requirements Specification* (Especificación de Requerimientos de Software).
- **RF**: Requerimiento Funcional.
- **RNF**: Requerimiento No Funcional.
- **RGB**: *Red, Green, Blue* (Rojo, Verde, Azul), modelo de color estándar de las imágenes digitales.
- **ViT**: *Vision Transformer*.
- **GPU**: *Graphics Processing Unit*.
- **CPU**: *Central Processing Unit*.
- **EARS**: *Easy Approach to Requirements Syntax*.
- **cm**: centímetros, unidad de medida de la distancia de desplazamiento.
- **ISO/IEC/IEEE**: *International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers*.

II. Referencias

Este documento hace referencia a los siguientes documentos y fuentes técnicas: el estándar ISO/IEC/IEEE 29148:2018 para la ingeniería de requerimientos de sistemas y software [1]; el artículo que introduce la sintaxis EARS para la redacción de requerimientos [2]; el modelo de estimación de profundidad monocular Depth Anything [3] y su antecesor MiDaS [4]; los modelos de segmentación semántica basados en *transformers* SegFormer [5] y Mask2Former [6]; y el *framework* de aprendizaje profundo PyTorch [7], sobre el cual se ejecutarán los modelos anteriores. Las entradas bibliográficas completas se listan en la sección de referencias al final de este documento.

III. Requisitos

III-A. Funciones

Los requerimientos funcionales del sistema se redactan utilizando el enfoque EARS (*Easy Approach to Requirements Syntax*) [2], el cual clasifica los requerimientos en ubicuos, activados por evento, activados por estado y de manejo de comportamientos indeseados. Cada requerimiento cuenta con un identificador único asignado manualmente.

[RF-001] El sistema deberá aceptar como entrada dos imágenes RGB monoculares de una misma escena interior, correspondientes a los estados “antes” y “después”.

[RF-002] Cuando la persona usuaria cargue la escena “antes” y la escena “después”, el sistema deberá ejecutar el modelo de estimación de profundidad monocular sobre ambas imágenes para obtener sus respectivos mapas de profundidad.

[RF-003] El sistema deberá comparar los dos mapas de profundidad generados para determinar si existen diferencias significativas entre la escena “antes” y la escena “después”.

[RF-004] Cuando se detecte una diferencia significativa entre los mapas de profundidad, el sistema deberá clasificar dicha diferencia como *desaparición de objeto* o como *desplazamiento de objeto*.

[RF-005] Cuando la diferencia detectada se clasifique como desplazamiento de objeto, el sistema deberá calcular la distancia de desplazamiento en centímetros, a partir de la información de profundidad estimada.

[RF-006] Cuando se detecte una diferencia entre las dos escenas, el sistema deberá ejecutar el modelo de segmentación semántica sobre la región de la imagen donde ocurrió el cambio, con el fin de identificar la categoría del objeto involucrado.

[RF-007] El sistema deberá restringir el reconocimiento de objetos a categorías correspondientes a ambientes interiores.

[RF-008] El sistema deberá presentar a la persona usuaria un reporte que indique si se detectó un cambio, el tipo de cambio (desaparición o desplazamiento), la categoría del objeto involucrado y, cuando corresponda, la distancia de desplazamiento calculada en centímetros.

[RF-009] Mientras el sistema se encuentre ejecutando la inferencia de los modelos de estimación de profundidad o de segmentación semántica, el software deberá priorizar dicha ejecución sobre el hardware de aceleración por GPU disponible.

[RF-010] Si no existe una GPU disponible en el equipo donde se ejecuta el sistema, el software deberá ejecutar la inferencia de los modelos sobre la CPU y notificar a la persona usuaria que el procesamiento podría tardar más tiempo de lo habitual.

[RF-011] Si no se detecta ninguna diferencia significativa entre los mapas de profundidad de las dos escenas, el sistema deberá informar a la persona usuaria que no se identificaron cambios.

[RF-012] Si una de las imágenes de entrada no corresponde a una escena interior válida o se encuentra corrupta, el sistema deberá rechazar el procesamiento y emitir un mensaje de error descriptivo a la persona usuaria.

[RF-013] Si la estimación de profundidad monocular falla debido a condiciones de baja iluminación en alguna de las imágenes de entrada, el sistema deberá emitir una advertencia a la persona usuaria indicando que el resultado puede no ser confiable.

[RF-014] El sistema deberá permitir a la persona usuaria consultar el historial de comparaciones realizadas durante la sesión de uso activa.

III-B. Requisitos de usabilidad

[RNF-001] El sistema deberá presentar los resultados de la comparación (tipo de cambio, categoría del objeto y distancia de desplazamiento) en un lenguaje no técnico, comprensible para personas usuarias sin conocimientos de visión por computadora.

[RNF-002] El sistema deberá indicar visualmente, mediante resaltado sobre la imagen, la región de la escena donde se detectó el cambio.

[RNF-003] El sistema deberá completar el flujo de carga de imágenes, ejecución y visualización del resultado en un máximo de tres interacciones por parte de la persona usuaria.

III-C. Interfaces externas

[RI-001] El sistema deberá proveer una interfaz de entrada para la carga de las dos imágenes RGB monoculares, ya sea mediante un archivo almacenado en el equipo o mediante una captura directa desde una cámara conectada.

[RI-002] El sistema deberá utilizar internamente el *framework* PyTorch [7] como interfaz de ejecución hacia los modelos preentrenados de estimación de profundidad monocular y de segmentación semántica.

[RI-003] El sistema deberá proveer una interfaz de salida que presente el reporte de resultados en pantalla, incluyendo las imágenes originales, el resaltado del cambio detectado y el texto descriptivo del resultado.

III-D. Requisitos de rendimiento

[RR-001] El sistema deberá completar el procesamiento completo de un par de imágenes (estimación de profundidad, comparación y segmentación semántica) en un tiempo no mayor a 15 segundos cuando se ejecute sobre una GPU dedicada.

[RR-002] El sistema deberá admitir imágenes de entrada con una resolución mínima de 640×480 píxeles sin degradar la precisión de la estimación de profundidad por debajo de los valores reportados por el modelo preentrenado utilizado.

III-E. Requisitos de la base de datos lógica

No aplica. El sistema no requiere una base de datos relacional o estructurada, ya que su función se limita al procesamiento de un par de imágenes por sesión y a la generación de un reporte inmediato; el historial de comparaciones referido en el requerimiento [RF-014] se mantiene en memoria durante la ejecución de la sesión activa, sin necesidad de persistencia estructurada en un motor de base de datos.

III-F. Restricciones de diseño

[RD-001] El sistema deberá implementarse en el lenguaje de programación Python.

[RD-002] El sistema deberá utilizar el *framework* de aprendizaje profundo PyTorch [7] para la ejecución de los modelos de estimación de profundidad monocular y de segmentación semántica.

[RD-003] El sistema deberá emplear modelos preentrenados de estimación de profundidad monocular, tales como Depth Anything [3] o MiDaS [4], sin requerir reentrenamiento desde cero.

[RD-004] El sistema deberá emplear un Vision Transformer preentrenado para segmentación semántica, tal como SegFormer [5] o Mask2Former [6].

[RD-005] El sistema deberá diseñarse para operar exclusivamente con una cámara RGB monocular estándar, sin depender de sensores de profundidad dedicados.

III-G. Atributos del sistema de software

[RA-001] *Confiabilidad*: El sistema deberá informar explícitamente a la persona usuaria cuando las condiciones de la imagen de entrada (iluminación, resolución) puedan comprometer la confiabilidad del resultado, conforme a lo indicado en el requerimiento [RF-013].

[RA-002] *Portabilidad*: El sistema deberá poder ejecutarse en cualquier equipo que cuente con Python y las bibliotecas de PyTorch instaladas, independientemente de contar o no con una GPU, conforme a los requerimientos [RF-009] y [RF-010].

[RA-003] *Mantenibilidad*: El sistema deberá estructurarse en módulos independientes para la captura, la estimación de profundidad, la segmentación semántica y la lógica de comparación, de manera que cada modelo de aprendizaje profundo pueda sustituirse por una versión más reciente sin modificar los demás módulos.

III-H. Información de soporte

Los siguientes requerimientos de información de soporte (RIS) especifican los materiales complementarios que el sistema deberá proveer para facilitar su correcta operación, demostración y calibración técnica.

[RIS-001] El sistema deberá proveer un manual de usuario digital integrado que detalle las pautas recomendadas para la alineación física y posicionamiento de la cámara RGB monocular, con el fin de garantizar la consistencia espacial entre la toma "antes" y "después".

[RIS-002] El sistema deberá incluir un conjunto de imágenes de prueba precargadas de espacios interiores comunes para permitir la demostración y verificación offline de las funciones de inferencia de profundidad y segmentación.

[RIS-003] El sistema deberá documentar de forma clara los requisitos de iluminación y resolución mínima requeridos en las escenas de entrada, con el fin de mitigar el margen de error de los transformers de estimación.

IV. Verificación

El Cuadro I presenta, para la totalidad de los requerimientos definidos en la sección 3 (funciones, usabilidad, interfaces externas, rendimiento, restricciones de diseño, atributos del sistema de software e información de soporte), el método de verificación asociado (Inspección – I, Demostración – D, Análisis – A, Prueba – T) y una descripción del caso de verificación correspondiente.

V. Apéndices

V-A. Supuestos y dependencias

La elaboración de este documento asume lo siguiente:

- Se dispondrá de acceso a modelos preentrenados de estimación de profundidad monocular y de segmentación semántica de uso público (Depth Anything, MiDaS, SegFormer, Mask2Former), sin necesidad de entrenarlos desde cero.
- El equipo de desarrollo y ejecución cuenta con una GPU compatible con PyTorch para las pruebas de rendimiento descritas en la sección 3.4.
- Las escenas de prueba corresponden a ambientes interiores controlados, sin condiciones climáticas o de iluminación exterior variable.
- La cámara monocular utilizada para las capturas de prueba entrega imágenes en formato RGB estándar, sin distorsión óptica significativa.

V-B. Matriz de trazabilidad

No aplica. El presente proyecto no se deriva de un documento de requerimientos de nivel superior ni de un sistema preexistente del cual este producto constituya un subconjunto; los requerimientos aquí definidos se originan directamente en la ficha de presentación del proyecto, por lo que no existe una matriz de trazabilidad hacia requerimientos de orden superior.

Aprobación de la persona supervisora

En la Figura 2 se muestra la evidencia de la aprobación de este documento por parte de la persona supervisora del proyecto, Luis Alberto Chavarría Zamora.

Referencias

[1] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 – Systems and software engineering – Life cycle processes – Requirements engineering*. Geneva, Switzerland: International Organization for Standardization, 2018.

[2] A. Mavin, P. Wilkinson, A. Harwood, and M. Novak, "Easy approach to requirements syntax (EARS)," in *Proc. 17th IEEE Int. Requirements Eng. Conf. (RE'09)*, Atlanta, GA, USA, 2009, pp. 317–322.

[3] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth Anything: Unleashing the power of large-scale unlabeled data," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

[4] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, 2022.

[5] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 12077–12090.

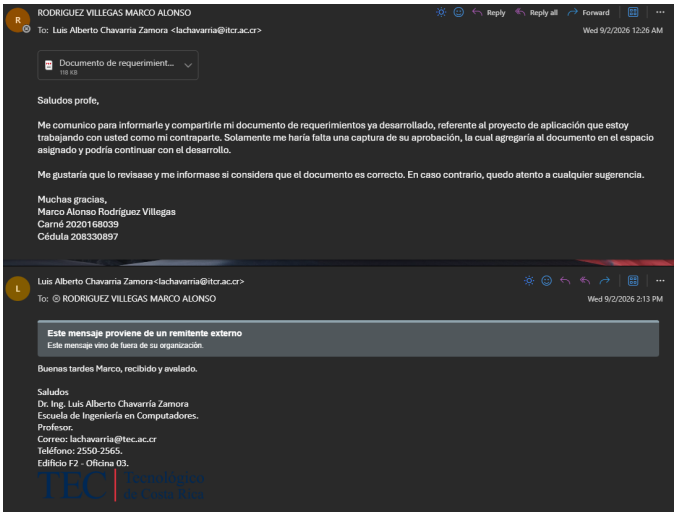


Figura 2. Evidencia de aprobación del documento de requerimientos.

[6] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar, "Masked-attention mask transformer for universal image segmentation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 1290–1299.

[7] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

Cuadro I
MATRIZ DE VERIFICACIÓN DE REQUERIMIENTOS DEL SISTEMA

ID	Método	Descripción del caso de verificación
RF-001	I	Inspeccionar que el módulo de carga acepte dos archivos de imagen RGB y los almacene correctamente en memoria antes de la inferencia.
RF-002	T	Ejecutar el sistema con un par de imágenes de prueba y verificar que se generen dos mapas de profundidad, uno por cada escena.
RF-003	T	Suministrar pares de imágenes con y sin cambios conocidos y verificar que el sistema detecte correctamente la presencia o ausencia de diferencias.
RF-004	T	Suministrar casos de prueba etiquetados como “desaparición” y como “desplazamiento” y verificar que el sistema los clasifique de forma correcta.
RF-005	A	Comparar la distancia de desplazamiento calculada por el sistema contra una medición de referencia tomada manualmente en la escena física, verificando que el error relativo se encuentre dentro de un margen aceptable.
RF-006	D	Demostrar, con un caso de cambio conocido, que el sistema ejecuta el modelo de segmentación semántica sobre la región afectada y devuelve una categoría de objeto.
RF-007	I	Inspeccionar el conjunto de categorías reconocidas por el modelo de segmentación configurado y confirmar que corresponden a objetos de ambientes interiores.
RF-008	D	Demostrar, ante un caso de prueba completo, que el reporte final incluye el tipo de cambio, la categoría del objeto y la distancia cuando corresponda.
RF-009	T	Ejecutar el sistema en un equipo con GPU disponible y verificar, mediante herramientas de monitoreo de hardware, que la inferencia se ejecuta sobre la GPU.
RF-010	T	Ejecutar el sistema en un equipo sin GPU disponible y verificar que la inferencia se complete sobre CPU junto con la notificación correspondiente a la persona usuaria.
RF-011	T	Suministrar dos imágenes idénticas o sin cambios y verificar que el sistema reporte la ausencia de diferencias.
RF-012	T	Suministrar un archivo de imagen corrupto o no perteneciente a un ambiente interior y verificar que el sistema rechace el procesamiento con un mensaje de error.
RF-013	T	Suministrar una imagen capturada en condiciones de baja iluminación y verificar que el sistema emita la advertencia correspondiente.
RF-014	D	Demostrar que, tras realizar varias comparaciones en una misma sesión, la persona usuaria puede consultar el historial de resultados generados.
RNF-001	D	Demostrar que los resultados en pantalla se presentan utilizando terminología comprensible y no técnica para la persona usuaria final.
RNF-002	D	Cargar un par de imágenes con diferencias y verificar visualmente en la interfaz que la región modificada se resalta con una caja o máscara de color.
RNF-003	D	Demostrar que el flujo completo, desde la carga de las imágenes hasta la visualización del reporte, se realiza en un máximo de tres interacciones físicas de la persona usuaria.
RI-001	D	Demostrar que la interfaz de usuario provee un control interactivo funcional para seleccionar y cargar dos archivos de imagen RGB monoculares.
RI-002	I	Inspeccionar el código fuente para confirmar la inicialización e interacción con las APIs del *framework* PyTorch para la inferencia de modelos.
RI-003	D	Confirmar visualmente que el reporte desplegado en la pantalla de salida integra simultáneamente las imágenes de entrada, el área resaltada y el texto con los resultados del análisis.
RR-001	T	Ejecutar el pipeline de procesamiento con un par de imágenes de prueba sobre hardware con GPU dedicada compatible y medir, con cronómetro de software, que el tiempo total de ejecución sea menor o igual a 15 segundos.
RR-002	T	Suministrar imágenes de entrada con resolución exacta de 640×480 píxeles y evaluar que el mapa de profundidad monocular resultante no sufra distorsiones de escala severas.
RD-001	I	Inspeccionar el repositorio de código fuente para validar que la implementación total del software se haya desarrollado en el lenguaje Python.
RD-002	I	Inspeccionar el archivo de dependencias del proyecto (`requirements.txt` o similar) para confirmar el uso del *framework* PyTorch.
RD-003	I	Verificar, en el script de carga de modelos, que se importan los pesos preentrenados de Depth Anything o MiDaS, sin rutinas de entrenamiento desde cero.
RD-004	I	Verificar en el código fuente que la segmentación de objetos se ejecuta mediante la importación de un Vision Transformer preentrenado (SegFormer o Mask2Former).
RD-005	I	Inspeccionar el diagrama de arquitectura y el código para validar que la única fuente de datos de entrada espacial es una secuencia o archivo de imágenes RGB monoculares.
RA-001	T	Alimentar el sistema con una imagen de prueba capturada en oscuridad extrema y verificar que el software detecte la condición adversa y despliegue un mensaje de alerta de confiabilidad.
RA-002	T	Ejecutar secuencialmente el software en un entorno con aceleración GPU habilitada y en un entorno puramente CPU, comprobando que la aplicación complete la ejecución en ambos casos.
RA-003	I	Inspeccionar el diseño modular del código fuente para constatar que el pipeline de captura, el de estimación de profundidad y el de segmentación se encuentran desacoplados en archivos o clases independientes.
RIS-001	I	Inspeccionar el manual de usuario digital integrado y confirmar que incluye pautas claras de alineación y posicionamiento de la cámara para garantizar la consistencia espacial entre capturas.
RIS-002	I	Inspeccionar el conjunto de imágenes de prueba precargadas y confirmar que permite ejecutar de forma offline las funciones de inferencia de profundidad y segmentación semántica.
RIS-003	I	Inspeccionar la documentación de requisitos de iluminación y resolución mínima y confirmar que describe explícitamente los umbrales recomendados para las escenas de entrada.